



Universidad Andrés Bello

Magíster en Ciencia de Datos e Inteligencia Artificial

MCDI500

Programación para la Ciencia de Datos

## **Evaluación Sumativa 2**

Avance Fase 3

Semana 2

Preparación reproducible de órdenes de Compra Ágil de Valparaíso

### **Grupo 7**

Benjamín Araya Matta

Alan Cañete Calquin

Patricio Cortez Triviño

Jorge Guaico Ortega

Docente: Dr. Omar Salinas Silva Entrega

original: 25 de septiembre de 2026

Revisión técnica: 2026-09-27

<https://github.com/BenjaArayaM/proyecto-grupo7-mcdi500>

## **Contenido**

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Codificación funcional y arquitectura básica</b> | <b>3</b> |
| <b>2</b> | <b>Preprocesamiento y transformación</b>            | <b>3</b> |
| <b>3</b> | <b>Validación técnica y verificación</b>            | <b>4</b> |
| <b>4</b> | <b>Eficiencia y optimización</b>                    | <b>5</b> |
| <b>5</b> | <b>Diseño estructurado del código</b>               | <b>7</b> |
| <b>6</b> | <b>Programación Orientada a Objetos - POO</b>       | <b>8</b> |
| <b>7</b> | <b>Documentacion de Arquitectura</b>                | <b>8</b> |
| <b>8</b> | <b>Repositorio GitHub</b>                           | <b>9</b> |
| <b>9</b> | <b>Bibliografía</b>                                 |          |

## **1 Codificación funcional y arquitectura básica**

La revisión conserva las dos estrategias evaluadas: un recorrido con bucle y una selección vectorizada con pandas. Ambas reciben una serie de montos y dos límites, y devuelven una lista con los valores estrictamente fuera del intervalo, en su orden original y con sus repeticiones. El uso de estructuras tabulares se apoya en el enfoque de análisis de datos presentado por McKinney (2010); la elección de la estrategia se decide aquí mediante mediciones.

La entrega anterior dejaba parte de la carga, validación y cálculo en el notebook. En esta revisión se separan las responsabilidades en módulos reutilizables y se incorpora una arquitectura orientada a objetos para el pipeline de procesamiento. Las funciones estadísticas del núcleo IQR se mantienen sin estado, ya que permiten comparar directamente las estrategias de filtrado y comparten el cálculo de límites mediante `calcular_limites_iqr`.

El procesamiento general se organiza mediante `PipelineCompraAgil`, que coordina la carga, validación, transformación, generación de métricas y exportación. `TransformadorDatos` mantiene la configuración de transformación; `GeneradorMetricas` concentra el cálculo de métricas; y `ExportadorResultados` separa la generación de archivos de las demás responsabilidades. El tratamiento de faltantes utiliza la interfaz `EstrategiaFaltantes` y sus implementaciones concretas, aplicando herencia y polimorfismo. Esta separación permite probar responsabilidades individuales y evita que la lógica de procesamiento quede acoplada al notebook.

Tabla 1: Componentes implementados y evidencia verificable

| Componente                  | Responsabilidad                                               |
|-----------------------------|---------------------------------------------------------------|
| Src/datos_iqr.py            | Localización, lectura y validación de ordenes procesadas      |
| Src/funciones_iqr.py        | Validación de serie, limites compartidos y dos filtros        |
| Src/benchmark_iqr.py        | Replicas de carga. Tiempo, memoria, e intervalos de cambio    |
| Tests/test_funciones_iqr.py | Pruebas ejecutables de contratos, casos limites e integración |
| Notebooks                   | Ejecución, tablas, gráficos e interpretación                  |
| Docs y evidencias           | Métricas, muestras, informe, pruebas y procedencias           |

Rutas relativas a la carpeta F3.

## 2 Preprocesamiento y transformación

La entrada sigue siendo F2/data/processed/ordenes.csv. F2 transforma el archivo original de 11.177 filas y 63 columnas en 541 órdenes con atributos de cabecera consistentes. F3 no repite la limpieza, no trata las filas exportadas como órdenes independientes y no absorbe las reglas monetarias o de faltantes de F2.

Se seleccionan el código de orden, MontoNetoOC\_CLP y la marca IQR de F2. La conversión a número utiliza control estricto de errores. Un valor no interpretable, faltante o infinito detiene el análisis; no se convierte silenciosamente en cero ni se elimina. Los códigos deben ser no vacíos y únicos. En la base comprobada existen 541 montos válidos y cero faltantes.

Se mantienen todas las monedas y estados porque el monto neto en CLP es la variable comparable provista por F2. Se conservan las 61 órdenes extremas: una clasificación estadística no demuestra un error de origen. Se eliminan cero registros y se imputan cero valores. Las réplicas usadas después para medir carga no se incorporan a este conjunto analítico.

### 3 Validación técnica y verificación

Con interpolación lineal se obtiene  $Q1 = 217.923 \text{ CLP}$ ,  $Q3 = 1.620.152 \text{ CLP}$  e  $IQR = 1.402.229 \text{ CLP}$ . Con factor 1,5, el límite inferior es  $-1.885.420,50 \text{ CLP}$  y el superior es  $3.723.495,50 \text{ CLP}$ . La interpolación se declara explícitamente para hacer comparable el cálculo con F2 (pandas development team, 2026).

Las dos implementaciones producen la misma lista completa de 61 valores, no solo el mismo conteo. También coinciden las marcas fila por fila y los códigos de las órdenes identificadas. La prueba de integración contrasta el nuevo cálculo con `marcar_iqr` de F2 y con los resultados guardados por esa fase. Los límites y los datos de entrada permanecen intactos.

Tabla 2: Casos cubiertos por las pruebas automatizadas

| Caso                                     | Resultado Esperado                                           |
|------------------------------------------|--------------------------------------------------------------|
| Sin valores fuera de rango               | Lista vacía en ambos filtros.                                |
| Todos fuera de rango                     | Se conservan todos los valores; límites externos $[-1, 1]$ . |
| Serie vacía con límites conocidos        | Lista vacía; no se intentan estimar cuartiles.               |
| Estimación con serie vacía               | Error explícito: faltan observaciones para $Q1$ y $Q3$ .     |
| Valores iguales a los límites            | No son extremos; comparaciones estrictas.                    |
| Duplicados e índices no consecutivos     | Se conserva el orden y la multiplicidad de los valores.      |
| Serie constante o de un elemento         | IQR cero, sin extremos dentro de su intervalo.               |
| Faltantes, infinitos o códigos repetidos | Rechazo antes de medir; no hay corrección silenciosa.        |
| Integración con F2                       | Mismos límites, 541 OC y 61 extremos.                        |

Se ejecutaron 24 pruebas de F3 sin fallos, errores ni omisiones en el entorno declarado. Los casos parametrizados comprueban los dos filtros. La prueba de rendimiento valida la conservación de muestras y la equivalencia, sin imponer qué implementación debe ganar. Las evidencias están en F3/evidencias/pruebas\_formativa3.json y su registro de texto.

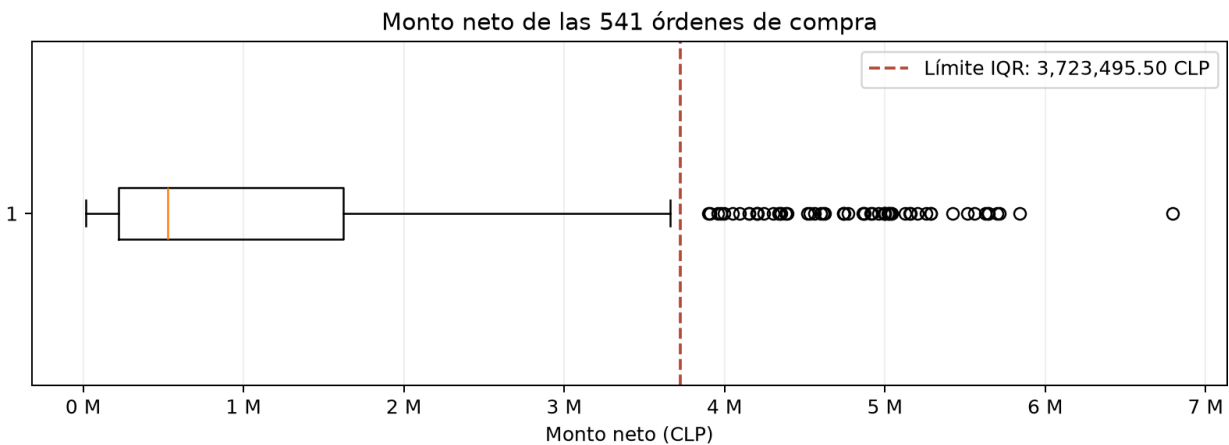


Figura 1: Distribución del monto neto de las 541 órdenes reales.

La línea discontinua señala el límite superior. Los puntos extremos se conservan en F2 y F3.

## 4 Eficiencia y optimización

### Protocolo temporal

Se preserva el procedimiento aprobado: `time.perf_counter`, 1.000 llamadas por medición, tres mediciones por estrategia y el menor tiempo observado. Se añaden tres llamadas de calentamiento y alternancia del orden de los métodos para reducir efectos del inicio y del orden de ejecución. El reloj se elige por su resolución para intervalos breves (Python Software Foundation, 2026a).

Se evalúan 541, 1.000, 2.000, 5.000, 10.000, 20.000 y 50.000 valores. Los tamaños mayores se construyen repitiendo cíclicamente la serie y truncándola. Son entradas sintéticas para medir escalamiento: no constituyen nuevas órdenes ni amplían la evidencia estadística del proyecto. Los límites obtenidos de las 541 OC permanecen fijos en todos los tamaños.

La lectura, la construcción de réplicas, la validación y los cuartiles se realizan fuera del intervalo cronometrado. Ambos métodos incluyen la creación de la lista final. Así se compara la misma tarea y el

mismo contrato de salida, manteniendo intactos los cuerpos de los dos filtros de la entrega. El CSV contiene mínimo, mediana y máximo por llamada; el JSON conserva las tres mediciones completas de cada combinación.

### Protocolo espacial

La memoria se mide en llamadas separadas, con tracemalloc desactivado durante la medición temporal. Para cada método y tamaño se registran tres picos incrementales y se presenta su mediana. La entrada existe antes de iniciar el rastreo y la lista de salida se mantiene viva al consultar el pico. La medida comprende las asignaciones rastreadas, no la RAM total del proceso ni necesariamente toda memoria nativa (Python Software Foundation, 2026b).

Tabla 3: Tiempo por llamada y memoria incremental del experimento actual

| Valores | Bucle ( $\mu$ s) | pandas ( $\mu$ s) | Bucle (KiB) | pandas (KiB) |
|---------|------------------|-------------------|-------------|--------------|
| 541     | 49,56            | 167,43            | 2,36        | 7,68         |
| 1.000   | 94,57            | 173,30            | 4,10        | 10,61        |
| 2.000   | 188,64           | 176,84            | 7,63        | 17,03        |
| 5.000   | 476,42           | 187,71            | 18,39       | 35,88        |
| 10.000  | 956,61           | 205,45            | 36,73       | 67,24        |
| 20.000  | 1.923,96         | 246,37            | 71,05       | 129,60       |
| 50.000  | 4.796,98         | 346,79            | 178,69      | 317,67       |

Tiempo: mínimo de tres mediciones de 1.000 llamadas. Memoria: mediana de tres picos separados;

1 KiB= 1.024 bytes. Solo la primera fila corresponde al tamaño real de la base.

**Entorno de estas mediciones:** Windows, Python 3.13.15, pandas 3.0.5 y NumPy 2.5.3. Con-texto: `revision_local_del_equipo`. Estas mediciones corresponden exclusivamente al entorno y al contexto declarados.

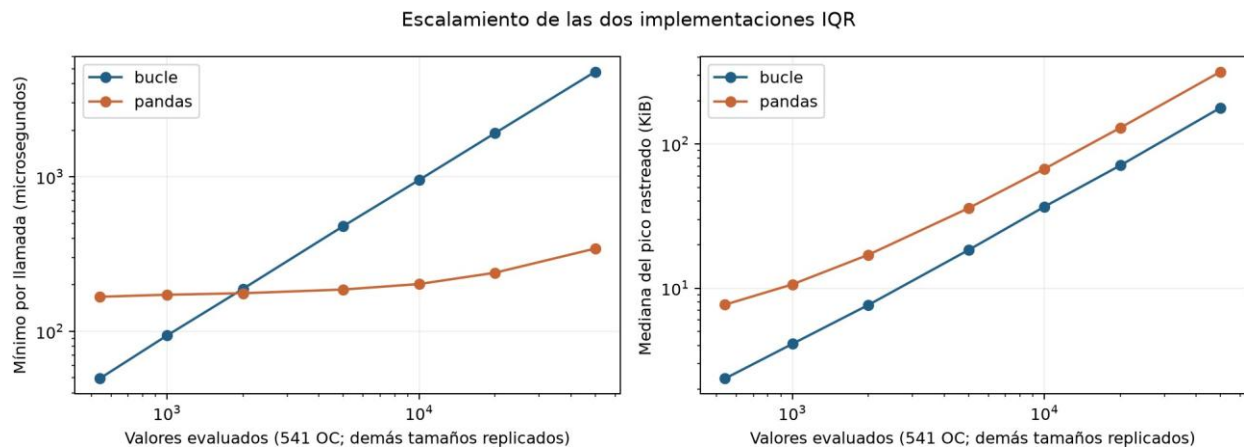


Figura 2: Escalamiento temporal y espacial en los tamaños medidos.

Ambos ejes usan escala logarítmica para mostrar tamaños pequeños y grandes. Las líneas unen mediciones y no constituyen un ajuste predictivo.

### **Interpretación y decisión**

En la entrega original se observaron aproximadamente 0,033 segundos para el bucle y 0,069 segundos para pandas, por 1.000 ejecuciones sobre 541 valores. Son datos históricos del notebook entregado y no se presentan como mediciones nuevas. En esta ejecución se obtienen 49,56 y 167,43 microsegundos por llamada, respectivamente; la alternativa más rápida en el tamaño nativo es bucle.

Pandas prepara comparaciones, construye máscaras booleanas, combina esas máscaras, selecciona la serie y convierte el resultado a lista. Ese costo fijo puede pesar más que el ahorro de recorrido en una entrada pequeña. Con más observaciones, el trabajo vectorizado puede compensarlo. Esta es una explicación del mecanismo compatible con las mediciones; no se afirma haber aislado cada operación interna mediante un perfilador.

El cambio de ventaja observado se ubica entre 1.000 y 2.000 valores. El muestreo permite acotar ese cambio, pero no establece el primer tamaño entero donde ocurre ni garantiza un umbral universal. En tamaños cercanos deben revisarse la variación entre repeticiones y el entorno de ejecución. Para el tamaño nativo medido, se mantiene el bucle como alternativa de referencia, conservando ambas estrategias para que la decisión pueda revisarse sin reescribir la lógica. Esta cautela es consistente con evaluaciones recientes de bibliotecas de DataFrame, donde el rendimiento relativo varía según las características de los datos y la etapa de preparación analizada (Mozzillo et al., 2025).

Los dos filtros tienen tiempo  $O(n)$ . El bucle requiere  $O(k)$  de memoria para su lista de salida; pandas añade máscaras y selección  $O(n)$ , además de la salida  $O(k)$ , donde  $k$  es la cantidad de extremos. Compartir orden asintótico no implica compartir constantes de tiempo o memoria. Estas cotas se refieren al filtrado, no al algoritmo interno de cuantiles ni a todo el pipeline.

## **5      Diseño estructurado del código**

Se mantiene el argumento aprobado sobre recursión. El problema consiste en recorrer una serie plana y comparar cada elemento con dos límites conocidos; tiene una secuencia de pasos definida y no requiere explorar estructuras anidadas de profundidad desconocida. La división funcional permite explicar, probar y cambiar cada paso sin añadir llamadas recursivas.

La decisión depende de la naturaleza del problema. Si posteriormente se incorporan jerarquías o procesos de profundidad variable, se reevaluará la recursión. No se agrega por cumplir una preferencia de estilo: la rúbrica admite división funcional y/o recursividad.

## **6 Programación Orientada a Objetos - POO**

La programación orientada a objetos se concentra en el pipeline de procesamiento. Utilizando el patrón de Strategy, permitió identificar el siguiente problema en la versión del pipeline que se encuentra en la carpeta F2; debido a la cantidad de datos faltantes, surge la necesidad de evaluar el impacto de diferentes formas de tratamiento de datos faltantes sobre un conjunto de ordenes de compra; el patrón elegido(Strategy), con las clases EstrategiaFaltantes, EstrategiaEliminarOrden, EstrategiaImputarModa y EstrategiaConservarMarca; se pudo solucionar este problema mediante el encapsulamiento de cada función de tratamiento de faltantes con su propia clase independiente, además haciendo que todas usen la misma interfaz. (Gamma, 1994)

Que hubiera pasado sin el patrón elegido, habría tenido que implementar la lógica interna de cada regla dentro de un bucle mediante condicionales fijas; además en caso de hacer cambios en una regla específica requerirían alterar la función `comparar_faltantes`, aumentando el riesgo de introducir errores.

```

class EstrategiaFaltantes(ABC):
    """Interfaz abstracta (Strategy) para el manejo de valores faltantes."""

    @property
    @abstractmethod
    def nombre(self) -> str:
        pass

    @abstractmethod
    def evaluar(self, ordenes: pd.DataFrame, columna: str, nulos: int) -> tuple[int, int]:
        """Retorna una tupla con (ordenes_retenidas, atributos_asignados)."""
        pass

Patricio Cortez, hace 3 días | autor (Patricio Cortez)
class EstrategiaEliminarOrden(EstrategiaFaltantes):
    @property
    def nombre(self) -> str:
        return "eliminar_orden_sin_dato"

    def evaluar(self, ordenes: pd.DataFrame, columna: str, nulos: int) -> tuple[int, int]:
        return len(ordenes) - nulos, 0

Patricio Cortez, hace 3 días | autor (Patricio Cortez)
class EstrategiaImputarModa(EstrategiaFaltantes):
    @property
    def nombre(self) -> str:
        return "imputar_moda"

    def evaluar(self, ordenes: pd.DataFrame, columna: str, nulos: int) -> tuple[int, int]:
        return len(ordenes), nulos

```

Figura 3: Patron Strategy con clase utilizadas

## Principios de la Programación Orientada a Objetos – POO:

1. Abstracción -> se crea una clase que usa la abstracción del manejo de faltantes

ocultando los detalles de implementación de cada regla, la clase se llama EstrategiasFaltantes(ABC)

```
class EstrategiaFaltantes(ABC):
    """Interfaz abstracta (Strategy) para el manejo de valores faltantes."""

    @property
    @abstractmethod
    def nombre(self) -> str:
        pass

    @abstractmethod
    def evaluar(self, ordenes: pd.DataFrame, columna: str, nulos: int) -> tuple[int, int]:
        """Retorna una tupla con (ordenes_retenidas, atributos_asignados)."""
        pass
```

Figura 4: Abstracción usada mediante la clase EstrategiaFaltantes(ABC)

2. Encapsulamiento -> las clases creadas agrupan los atributos y métodos,

centralizando el acceso a la lógica de estos.

```
class ExportadorResultados:
    def __init__(self, raiz: Path):
        self.raiz = raiz

    def exportar(self, resultado: dict) -> None:
        destinos = {
            "ordenes": "F2/data/processed/ordenes.csv",
            "relacion_rubros": "F2/data/processed/relacion_orden_rubro.csv",
            "variables": "F2/data/processed/variables_exploratorias.csv",
            "perfil_original": "F2/docs/perfil_original.csv",
            "comparacion_faltantes": "F2/docs/comparacion_faltantes.csv",
            "conciliacion": "F2/docs/conciliacion_firmas.csv",
            "excepciones_detalle": "F2/docs/excepciones_detalle.csv",
            "cruce_exportacion": "F2/docs/cruce_exportacion.csv",
        }
        for tabla, destino in destinos.items():
            ruta = self.raiz / destino
            ruta.parent.mkdir(parents=True, exist_ok=True)
            resultado[tabla].to_csv(ruta, index=False, encoding="utf-8", lineterminator="\n", date_format="%Y-%m-%d", float_format="%.8f")

        for clave, destino in [("metricas", "F2/docs/metricas.json"), ("parametros", "F2/data/processed/parametros_transformacion.json")]:
            (self.raiz / destino).write_text(json.dumps(resultado[clave], ensure_ascii=False, indent=2, allow_nan=False) + "\n", encoding="utf-8")
```

Figura 5: Encapsulamiento -> clase ExportadorResultados, con atributos y métodos internos, que pueden ser llamados en cualquier parte del código.

3. Herencia -> las clases heredan la estructura y el contrato de la clase base abstracta

```
class EstrategiaEliminarOrden(EstrategiaFaltantes):
    @property
    def nombre(self) -> str:
        return "eliminar_orden_sin_dato"

    def evaluar(self, ordenes: pd.DataFrame, columna: str, nulos: int) -> tuple[int, int]:
        return len(ordenes) - nulos, 0

Patricio Cortez, hace 3 días | autor (Patricio Cortez)
class EstrategiaImputarModa(EstrategiaFaltantes):
    @property
    def nombre(self) -> str:
        return "imputar_moda"

    def evaluar(self, ordenes: pd.DataFrame, columna: str, nulos: int) -> tuple[int, int]:
        return len(ordenes), nulos
```

Figura 6: Herencia, las clases EstrategiaEliminarOrden y EstrategiaImputarModa, heredan la estructura de la clase EstrategiaFaltantes

4. Polimorfismo -> El método evaluar se invoca de manera uniforme sobre distintos objetos, ejecutando la lógica propia de cada subclase sin importar cuál sea.

```
filas = []
for columna in columnas:
    nulos = int(ordenes[columna].isna().sum())
    for est in estrategias:
        retenidas, asignados = est.evaluar(ordenes, columna, nulos)
        filas.append({
            "columna": columna,
            "estrategia": est.nombre,
            "ordenes_iniciales": len(ordenes),
            "faltantes_originales": nulos,
            "ordenes_retenidas": retenidas,
            "atributos_reales_asignados_sin_evidencia": asignados
        })
return pd.DataFrame(filas)
```

Figura 7: Polimorfismo, dentro del método comparar\_faltantes.

Se descarta concentrar todo en el notebook porque impediría probar de forma aislada la lectura, los contratos y las responsabilidades del pipeline. También se descarta copiar F2 dentro de F3.

Se crean dos versiones de las reglas monetarias y de faltantes. F2 permanece como productor de datos procesados y F3 como consumidor y núcleo experimental, con el archivo procesado como interfaz entre ambas fases.

La salida de los filtros permanece como lista porque esa era la interfaz evaluada. Conserva valores, orden y multiplicidad, pero no el índice. Para sostener trazabilidad por orden se añade un contraste específico de máscaras y códigos, en vez de atribuir a la lista información que no contiene. El detalle de responsabilidades, alternativas y consecuencias está en F3/docs/arquitectura.md.

La evolución conserva el núcleo estadístico IQR como funciones sin estado, mientras que el pipeline de procesamiento utiliza objetos donde la configuración, la composición y las responsabilidades justifican su uso. Los datos sintéticos de rendimiento nunca se reutilizarán como nuevas observaciones de entrenamiento.

## **7 Documentación de arquitectura**

F2 conserva sus 17 funciones de preparación. F3 tenía inicialmente dos funciones de detección; esta revisión extiende esa base con validación, límites compartidos y un módulo de medición. La evolución responde a problemas concretos de la entrega: cuartiles repetibles fuera del notebook, entrada verificable y pruebas que se ejecutan directamente desde Python.

No se trasladan las reglas de F2 a un módulo nuevo. El archivo procesado y las métricas son la interfaz entre fases. La prueba contra `marcar_iqr` comprueba continuidad; la segunda implementación del criterio existe para comparar estrategias y no para redefinir la limpieza. Si en el futuro se extrae una utilidad compartida entre fases, esa refactorización requerirá pruebas de regresión de ambas para evitar cambios retrospectivos inadvertidos.

Las funciones estadísticas del núcleo IQR no necesitan mantener estado entre ejecuciones y, por ello, se conservan como funciones sin estado. Esta decisión es diferente del pipeline de procesamiento, donde sí resulta útil mantener configuración y responsabilidades explícitas mediante objetos.

La arquitectura POO permite separar responsabilidades y aplicar composición, herencia y polimorfismo donde aportan claridad. En particular, las estrategias de faltantes comparten una interfaz común y pueden ser sustituidas sin modificar el flujo que las utiliza. Esta combinación evita introducir una jerarquía artificial en el núcleo estadístico, pero permite estructurar el pipeline de procesamiento de acuerdo con las responsabilidades reales del proyecto (Universidad Andrés Bello, 2026a, 2026b).

- **Notebooks ejecutables**

El notebook corregido contiene once celdas de código, con narrativa para cada etapa y salidas conservadas. Además de la experimentación IQR, incorpora evidencia de la arquitectura POO, polimorfismo de las estrategias de faltantes e integración entre F2 y F3. El ejecutor inicia un proceso nuevo, verifica contadores consecutivos, detiene la ejecución ante errores y registra los hash del notebook, de los módulos y de las pruebas. La etiqueta de contexto y las versiones proceden de la ejecución registrada.

Se conserva la referencia al foro técnico incluida en la entrega: las decisiones de diseño se fundamentan en la naturaleza del problema y en evidencia de eficiencia, no en asumir que una estrategia gana siempre. La revisión amplía esa discusión con tamaños crecientes, memoria y contratos verificables. No se inventan nuevas intervenciones de integrantes en el foro.

La ejecución se reproduce con Python `scripts/ejecutar_formativa3.py`. El verificador comprueba que los datos y el código corresponden a las evidencias antes de compilar el PDF. Los tiempos pueden variar entre máquinas aunque las 541 OC, los límites y los 61 extremos deban permanecer consistentes. El informe se genera desde las métricas, evitando transcribir manualmente los resultados del benchmark.

## 8 Repositorio GitHub

Las correcciones de F3 fueron incorporadas progresivamente sobre la rama main, manteniendo commits específicos para documentar los cambios realizados. Entre las modificaciones se incluyen la actualización del README, la documentación de la arquitectura POO, la corrección de referencias históricas y la trazabilidad de las correcciones frente a la retroalimentación.

La arquitectura se documenta en F3/docs/arquitectura.md y las pruebas y ejecuciones quedan respaldadas mediante los archivos de F3/evidencias. El README describe el estado de las fases, la ejecución reproducible y la relación entre F2 y F3.

La revisión incorpora un archivo .mailmap para los pares de alias comprobados con el mismo correo, con el objetivo de mejorar la presentación de autoría histórica sin modificar los commits originales.

También se repararon por separado dos archivos de F2 que contenían marcas de conflicto: el notebook y el JSON de entorno. La reparación se mantuvo independiente de las modificaciones del código de preprocesamiento F2. Se conserva la variante histórica correspondiente al registro existente y no se presenta esa reparación como una nueva ejecución local de F2.

La fuente LaTeX se conserva como código de generación reproducible. El estado final del repositorio debe verificarse mediante el historial de commits, la ejecución de las pruebas y la compilación del informe antes de una entrega definitiva.

## 9 Bibliografías

Gamma, E. H. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

McKinney, W. (2010). Data structures for statistical computing in Python. En Proceedings of the 9th Python in Science Conference (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>

Mozzillo, A., Zecchini, L., Gagliardelli, L., Aslam, A., Bergamaschi, S. y Simonini, G. (2025).

Evaluation of Dataframe Libraries for Data Preparation on a Single Machine. En Proceedings of the 28th International Conference on Extending Database Technology, pp. 337–349.

<https://doi.org/10.48786/EDBT.2025.27>

pandas development team. (2026). pandas.Series.quantile. pandas documentation.

<https://pandas.pydata.org/docs/reference/api/pandas.Series.quantile.html>

Python Software Foundation. (2026b). tracemalloc — Trace memory allocations. Python documentation.

<https://docs.python.org/3/library/tracemalloc.html>

Universidad Andrés Bello. (2026b). MCDI500 Fase 3, Semana 2: Núcleo algorítmico, eficiencia y programación orientada a objetos [Material del curso]. Magíster en Ciencia de Datos e Inteligencia Artificial.

Universidad Andrés Bello. (2026a). MCDI500: Apunte Fase 3 [Material del curso]. Magíster en Ciencia de Datos e Inteligencia Artificial